2303A51832

B_26

AssignmentNumber:6.3(Present assignment number)/**24**(Total number of assignments)		
Q.No.	**Question**	***Expected Time to complete***
1	**Lab 6: AI-Based Code Completion – Classes, Loops, and Conditionals** **Lab Objectives** • To explore AI-powered auto-completion features for core Python constructs such as classes, loops, and conditional statements. • To analyze how AI tools suggest logic for object-oriented programming and control structures. • To evaluate the correctness, readability, and completeness of AI-generated Python code. **Lab Outcomes (LOs)** After completing this lab, students will be able to: • Use AI tools to generate and complete Python class definitions and methods. • Understand and assess AI-suggested loop constructs for iterative tasks. • Generate and evaluate conditional statements using AI-driven prompts. • Critically analyze AI-assisted code for correctness, clarity, and efficiency. **Task Description #1: Classes (Student Class)** **Scenario** You are developing a simple student information management module. **Task** • Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class. • The class should include attributes such as name, roll number, and branch. • Add a method display_details() to print student information. • Execute the code and verify the output. • Analyze the code generated by the AI tool for correctness and clarity. **Expected Output #1** • A Python class with a constructor (__init__) and a display_details() method. • Sample object creation and output displayed on the console. • Brief analysis of AI-generated code. **Prompt:**  **Code:**	Week3 - Wednesday

```python
class Student:
    """A class to represent a student."""

    def __init__(self, name, roll_number, branch):
        """
        Initialize student with name, roll number, and branch.

        Args:
            name (str): Student's name
            roll_number (int): Student's roll number
            branch (str): Student's branch of study
        """
        self.name = name
        self.roll_number = roll_number
        self.branch = branch

    def display_details(self):
        """Display all student information in a readable format."""
        print("-" * 40)
        print("Student Information")
        print("-" * 40)
        print(f"Name:        {self.name}")
        print(f"Roll Number: {self.roll_number}")
        print(f"Branch:      {self.branch}")
        print("-" * 40)


# Create Student objects and display their details
student1 = Student("Alice Johnson", 101, "Computer Science")
student1.display_details()

student2 = Student("Bob Smith", 102, "Electronics")
student2.display_details()
```

```
PS C:\Users\abbua\OneDrive\Desktop\AI ASSISTED CODING> & C:/Users/abbua/AppData/Local/Microsoft/WindowsApps/python3.11.exe "c:/Users/abbua/OneDrive/Desktop/AI ASSISTED CODING/AI assignment_6.3.py"
----------------------------------------
Student Information
----------------------------------------
Name:        Alice Johnson
Roll Number: 101
Branch:      Computer Science
----------------------------------------
----------------------------------------
Student Information
----------------------------------------
Name:        Bob Smith
Roll Number: 102
Branch:      Electronics
----------------------------------------
PS C:\Users\abbua\OneDrive\Desktop\AI ASSISTED CODING>
```

Explaination:

This code defines a Student class used to store and display basic student information. The class has a constructor (__init__) that initializes three attributes: name, roll number, and branch when a student object is created. The display_details() method neatly prints the student's information with clear labels and separators, making the output easy to read. At the end of the program, two Student objects are created with different details, and the display_details() method is called for each object to show their information on the console. Overall, the code is simple, well-structured, and clearly demonstrates how classes and objects work in Python.

Task Description #2: Loops (Multiples of a Number)

Scenario

You are writing a utility function to display multiples of a given number.

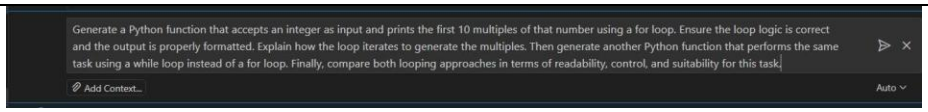
Task

- Prompt the AI tool to generate a function that prints the first 10 multiples of a given number using a loop.
- Analyze the generated loop logic.
- Ask the AI to generate the same functionality using another controlled looping structure (e.g., while instead of for).

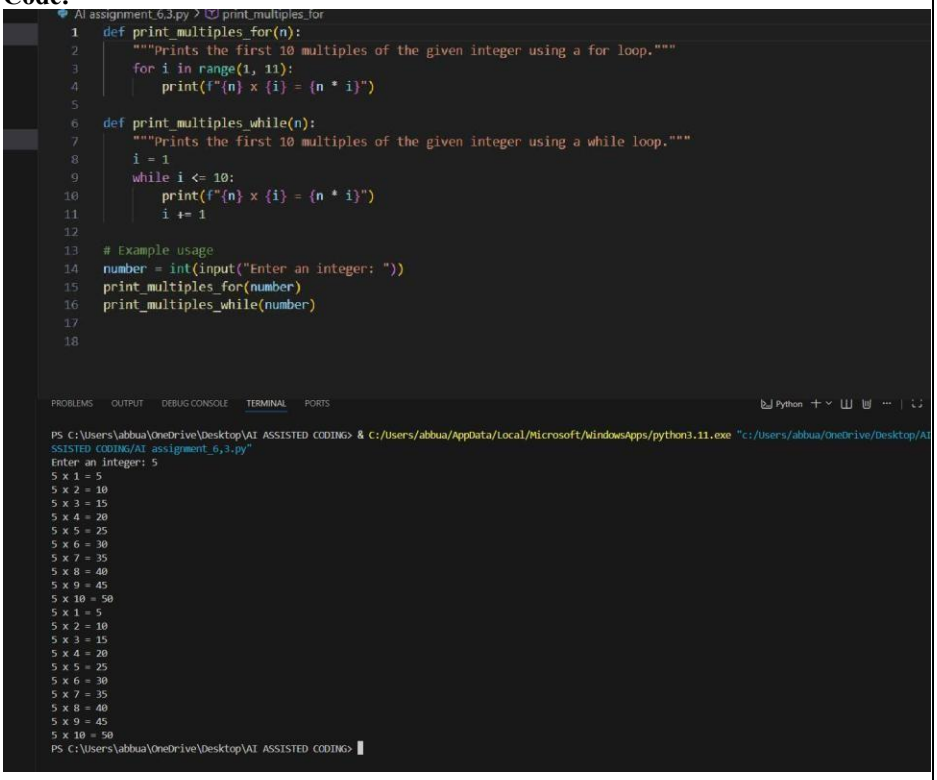
Expected Output #2

- Correct loop-based Python implementation.
- Output showing the first 10 multiples of a number.
- Comparison and analysis of different looping approaches.

Prompt:



Code:



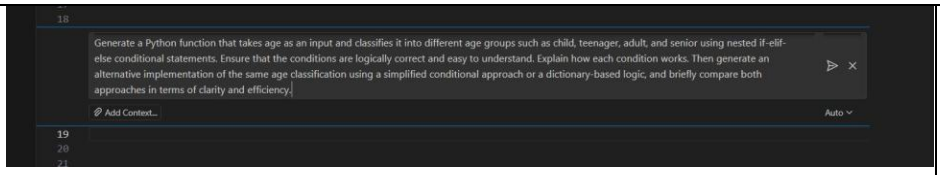
Explaination:
This code defines two functions to print the first 10 multiples of a given number using different looping structures. The print_multiples_for() function uses a for loop that runs from 1 to 10 and prints the multiplication result for each value. The print_multiples_while() function performs the same task using a while loop, where a counter variable starts at 1 and increments until it reaches 10. The program then asks the user to enter an integer and calls both functions, showing how the same logic can be implemented using both for and while loops in Python.

Task Description #3: Conditional Statements (Age Classification)

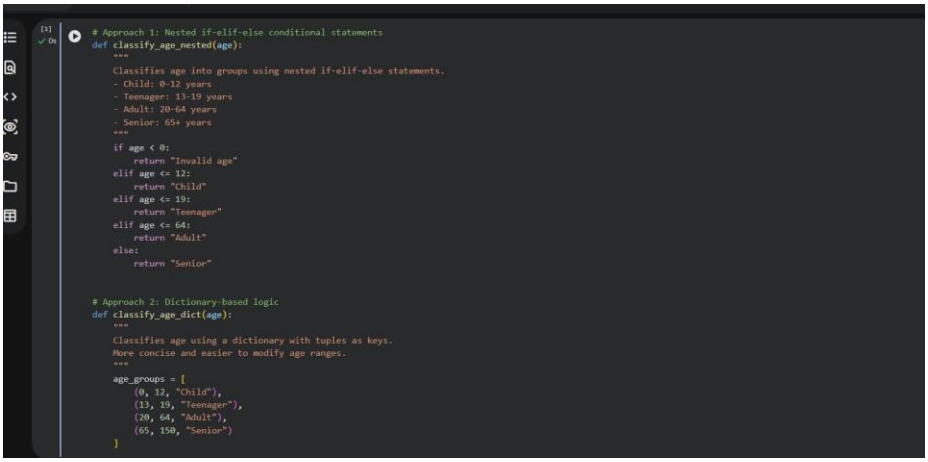
Scenario
You are building a basic classification system based on age.

Task
• Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups (e.g., child, teenager, adult, senior).
• Analyze the generated conditions and logic.
• Ask the AI to generate the same classification using alternative conditional structures (e.g., simplified conditions or dictionary-based logic).

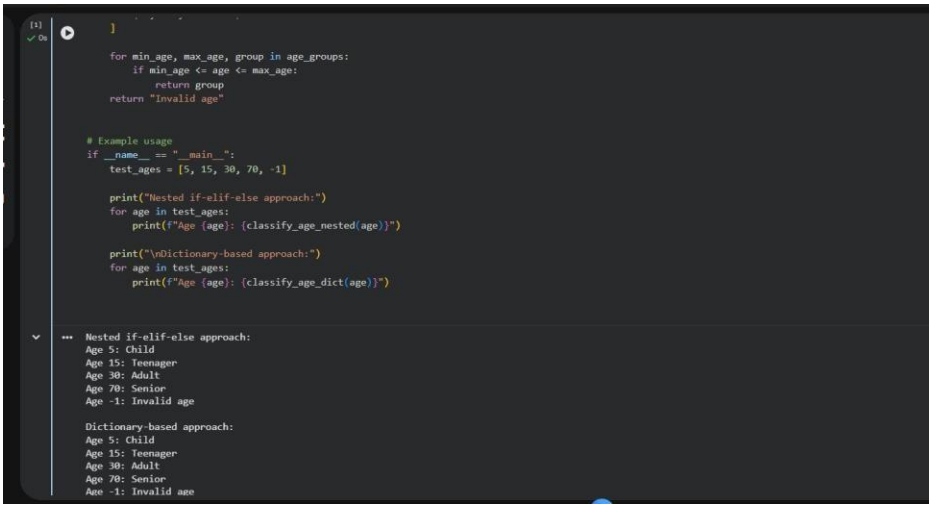
Expected Output #3
• A Python function that classifies age into appropriate groups.
• Clear and correct conditional logic.
• Explanation of how the conditions work.
Prompt:



Code:



```
# Approach 1: Nested if-elif-else conditional statements
def classify_age_nested(age):
    """
    Classifies age into groups using nested if-elif-else statements.
    - Child: 0-12 years
    - Teenager: 13-19 years
    - Adult: 20-64 years
    - Senior: 65+ years
    """
    if age < 0:
        return "Invalid age"
    elif age <= 12:
        return "Child"
    elif age <= 19:
        return "Teenager"
    elif age <= 64:
        return "Adult"
    else:
        return "Senior"


# Approach 2: Dictionary-based logic
def classify_age_dict(age):
    """
    Classifies age using a dictionary with tuples as keys.
    More concise and easier to modify age ranges.
    """
    age_groups = [
        (0, 12, "Child"),
        (13, 19, "Teenager"),
        (20, 64, "Adult"),
        (65, 150, "Senior")
    ]
```



```
    ]

    for min_age, max_age, group in age_groups:
        if min_age <= age <= max_age:
            return group
    return "Invalid age"


# Example usage
if __name__ == "__main__":
    test_ages = [5, 15, 30, 70, -1]

    print("Nested if-elif-else approach:")
    for age in test_ages:
        print(f"Age {age}: {classify_age_nested(age)}")

    print("\nDictionary-based approach:")
    for age in test_ages:
        print(f"Age {age}: {classify_age_dict(age)}")
```

```
Nested if-elif-else approach:
Age 5: Child
Age 15: Teenager
Age 30: Adult
Age 70: Senior
Age -1: Invalid age

Dictionary-based approach:
Age 5: Child
Age 15: Teenager
Age 30: Adult
Age 70: Senior
Age -1: Invalid age
```

Explaination:

This code shows two different ways to classify a person's age into groups such as child, teenager, adult, and senior. In the first approach, the classify_age_nested() function uses if-elif-else statements to check the age step by step and return the appropriate category, while also handling invalid negative ages. In the second approach, the classify_age_dict() function stores age ranges and their labels in a list of tuples and uses a loop to find where the given age fits, making the logic more compact and easier to modify. In the example usage section, a list of test ages is checked using both functions, and the results are printed to compare how each approach works.

Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario

You need to calculate the sum of the first n natural numbers.

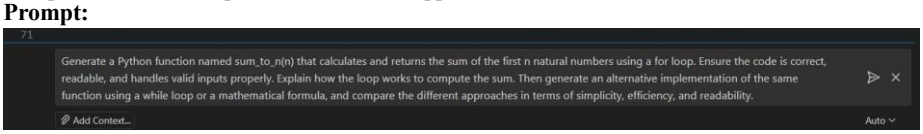
Task

• Use AI assistance to generate a sum_to_n() function using a for loop.

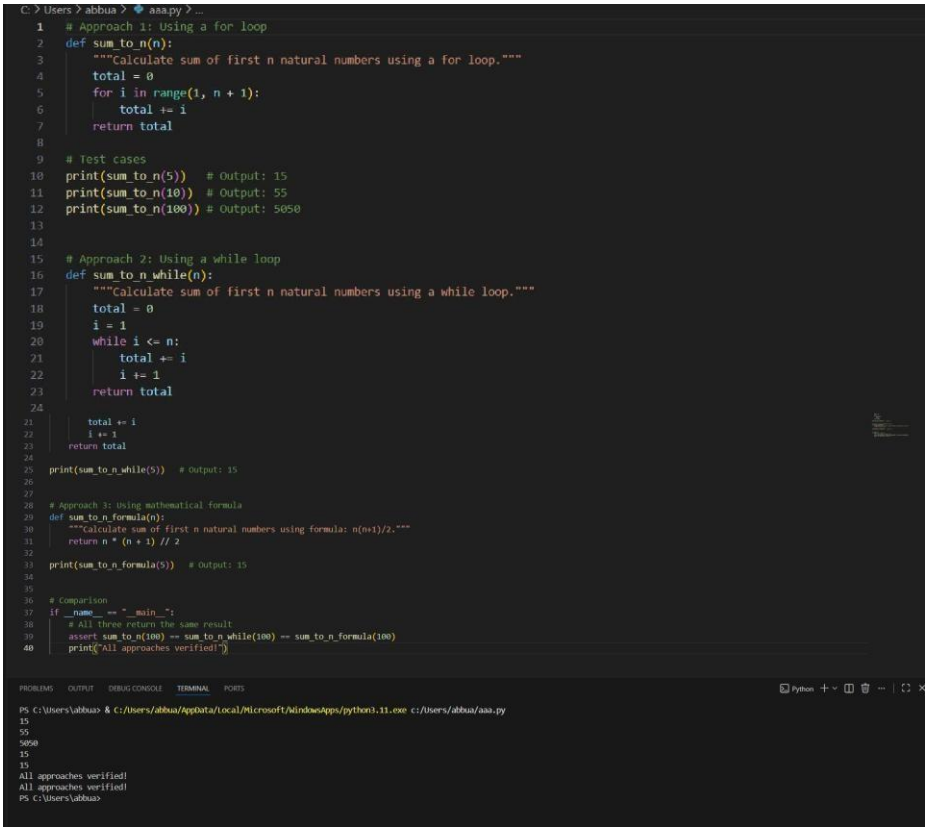
• Analyze the generated code.
• Ask the AI to suggest an alternative implementation using a while loop or a mathematical formula.

Expected Output #4
• Python function to compute the sum of first n numbers.
• Correct output for sample inputs.
• Explanation and comparison of different approaches.

Prompt:



Code:



Explaination:

This code shows **three different ways** to calculate the sum of the first *n* natural numbers. The first function uses a **for loop**, where numbers from 1 to *n* are added one by one to a total variable. The second function uses a **while loop**, which repeats the addition process until the counter reaches *n*. The third function uses a **mathematical formula** n(n+1)/2, which gives the result instantly without looping, making it the fastest method. Test cases check whether the functions give correct outputs, and the final assertion confirms that all three approaches produce the same result, proving their correctness.

Task Description #5: Classes (Bank Account Class)

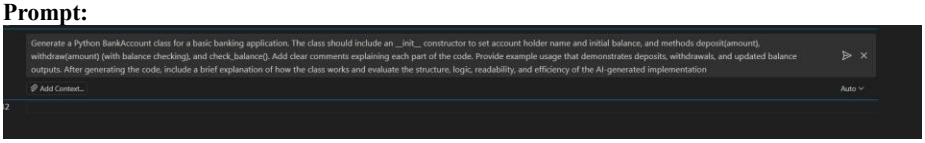
Scenario
You are designing a basic banking application.

Task
• Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(), and check_balance().

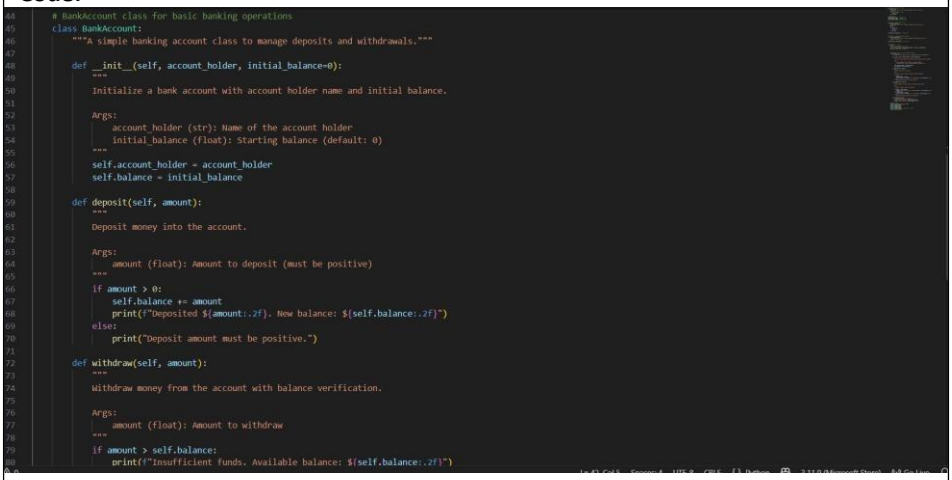
- Analyze the AI-generated class structure and logic.
- Add meaningful comments and explain the working of the code.

Expected Output #5
- Complete Python Bank Account class.
- Demonstration of deposit and withdrawal operations with updated balance.
- Well-commented code with a clear explanation.

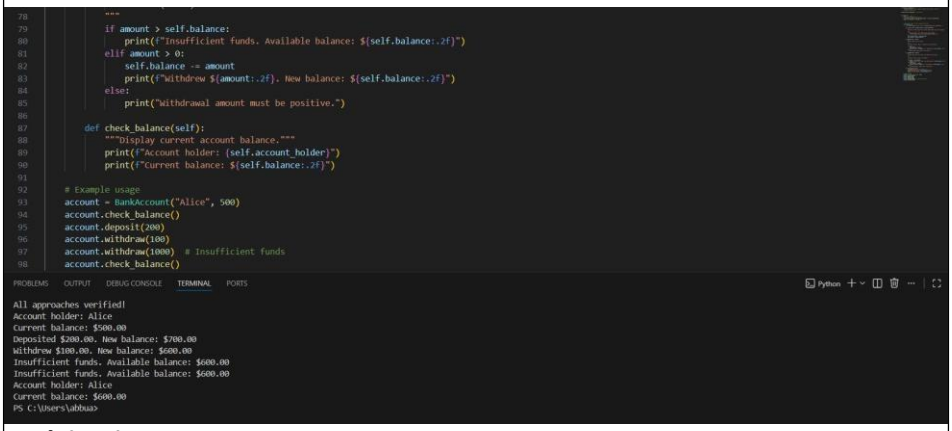
Prompt:



Code:



```python
# BankAccount class for basic banking operations
class BankAccount:
    """A simple banking account class to manage deposits and withdrawals."""

    def __init__(self, account_holder, initial_balance=0):
        """
        Initialize a bank account with account holder name and initial balance.

        Args:
            account_holder (str): Name of the account holder
            initial_balance (float): Starting balance (default: 0)
        """
        self.account_holder = account_holder
        self.balance = initial_balance

    def deposit(self, amount):
        """
        Deposit money into the account.

        Args:
            amount (float): Amount to deposit (must be positive)
        """
        if amount > 0:
            self.balance += amount
            print(f"Deposited ${amount:.2f}. New balance: ${self.balance:.2f}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        """
        Withdraw money from the account with balance verification.

        Args:
            amount (float): Amount to withdraw
        """
        if amount > self.balance:
            print(f"Insufficient funds. Available balance: ${self.balance:.2f}")
```



```python
        """
        if amount > self.balance:
            print(f"Insufficient funds. Available balance: ${self.balance:.2f}")
        elif amount > 0:
            self.balance -= amount
            print(f"Withdrew ${amount:.2f}. New balance: ${self.balance:.2f}")
        else:
            print("Withdrawal amount must be positive.")

    def check_balance(self):
        """Display current account balance."""
        print(f"Account holder: {self.account_holder}")
        print(f"Current balance: ${self.balance:.2f}")

# Example usage
account = BankAccount("Alice", 500)
account.check_balance()
account.deposit(200)
account.withdraw(100)
account.withdraw(1000)  # Insufficient funds
account.check_balance()
```

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
All approaches verified!
Account holder: Alice
Current balance: $500.00
Deposited $200.00. New balance: $700.00
Withdrew $100.00. New balance: $600.00
Insufficient funds. Available balance: $600.00
Insufficient funds. Available balance: $600.00
Account holder: Alice
Current balance: $600.00
PS C:\Users\abbaa>
```

Explaination:

This code defines a **BankAccount** class that simulates basic banking operations like depositing, withdrawing, and checking balance. The __init__method initializes the account holder's name and starting balance. The deposit() method adds money to the account only if the amount is positive, while the withdraw() method subtracts money after checking if there are enough funds and if the amount is valid. The check_balance() method displays the account holder's name and current balance. In the example usage, an account is created for "Alice" with an initial balance of 500, then deposits and withdrawals are performed, including a failed withdrawal due to insufficient funds, showing how the class manages transactions and updates the balance.